

# Micrositio de Pedidos y Delivery

---

Proyecto final · Kubernetes / DevOps

Sebastián Contreras · Grupo G-02

Capa de pedidos sobre ZUYU (SaaS de punto de venta para PyMEs)

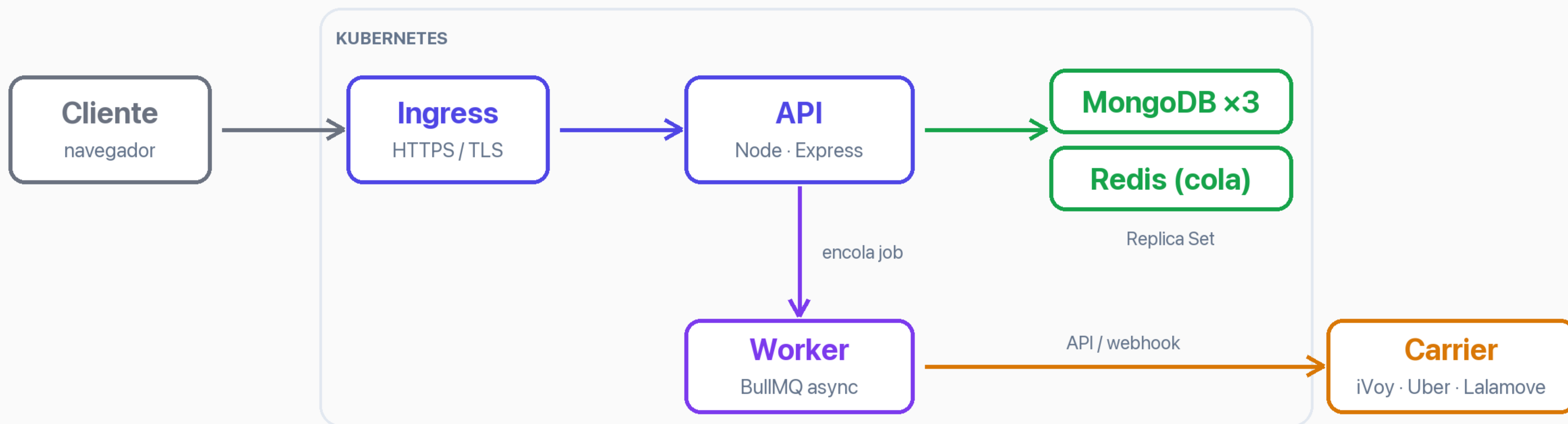
# El problema y la solución

- **Problema:** vender en línea y coordinar entregas sin montar infraestructura.
- **Solución:** una app web lista para PRODUCCIÓN, no solo “que funcione”.
- Catálogo → pedido → repartidor → entrega.

Alta disponibilidad · escalado automático · seguridad de red · CI/CD · observabilidad

CÓMO ESTÁ CONSTRUIDO

# Arquitectura

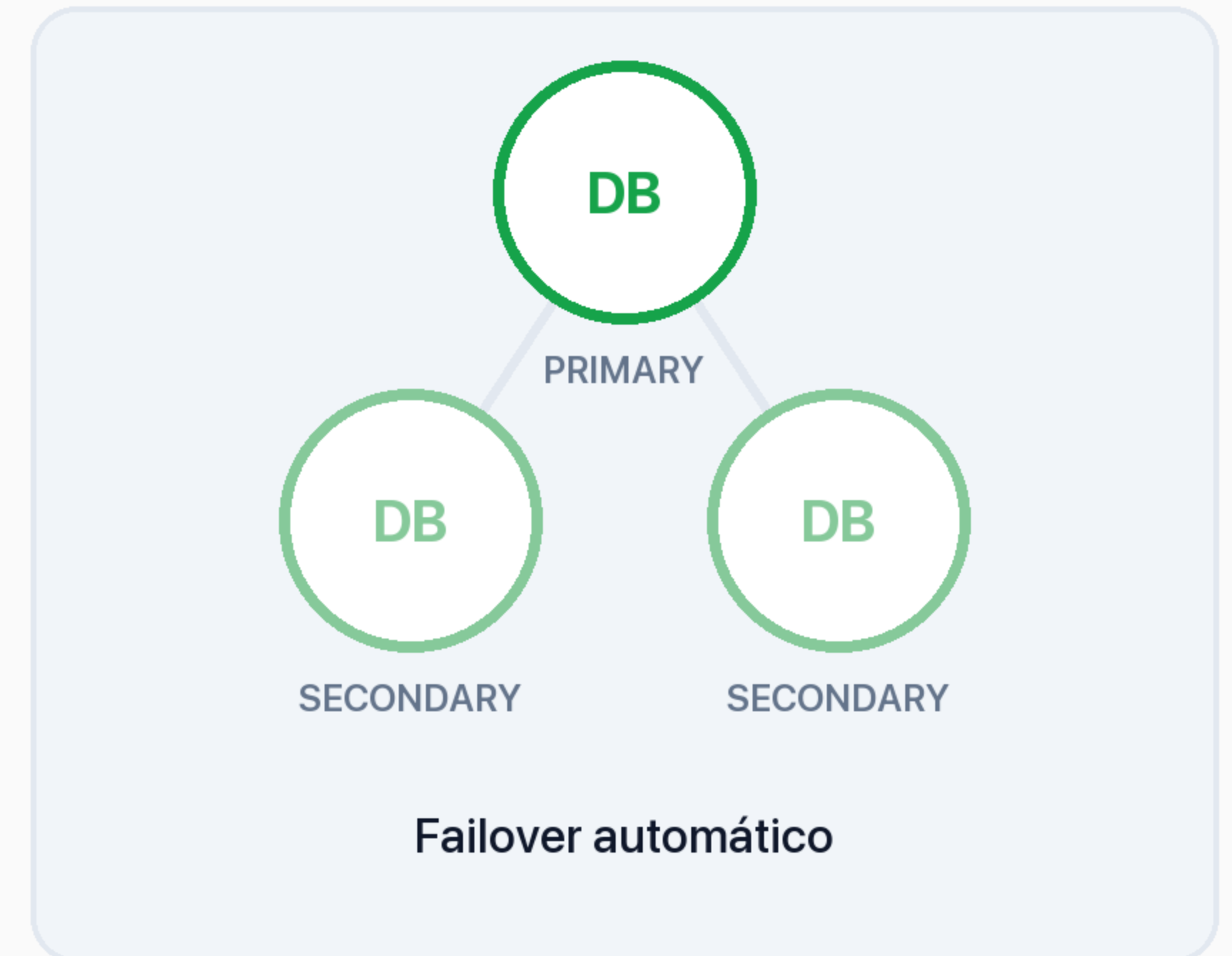


● Edge ● App ● Datos ● Worker async ● Externo



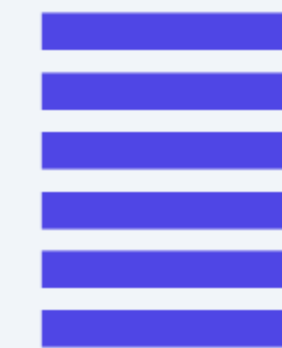
# MongoDB — Replica Set

- **Qué es:** 3 réplicas: 1 PRIMARY + 2 SECONDARY.
- **Cómo funciona:** si cae el PRIMARY, eligen otro solos.
- **Transacción atómica:** pedido + stock, todo o nada.
- **Por qué:** los pedidos son dinero.



# NetworkPolicy — firewall entre pods

- **Qué es:** reglas de quién habla con quién.
- **Estrategia:** default-deny + permitir lo justo.
- **Motor:** el CNI Calico la aplica.
- **Por qué:** frena el movimiento lateral.



## default-deny

todo bloqueado por defecto

✓ API → Mongo

✓ Ingress → API

☰ pod random → Mongo

# KEDA — escalado automático

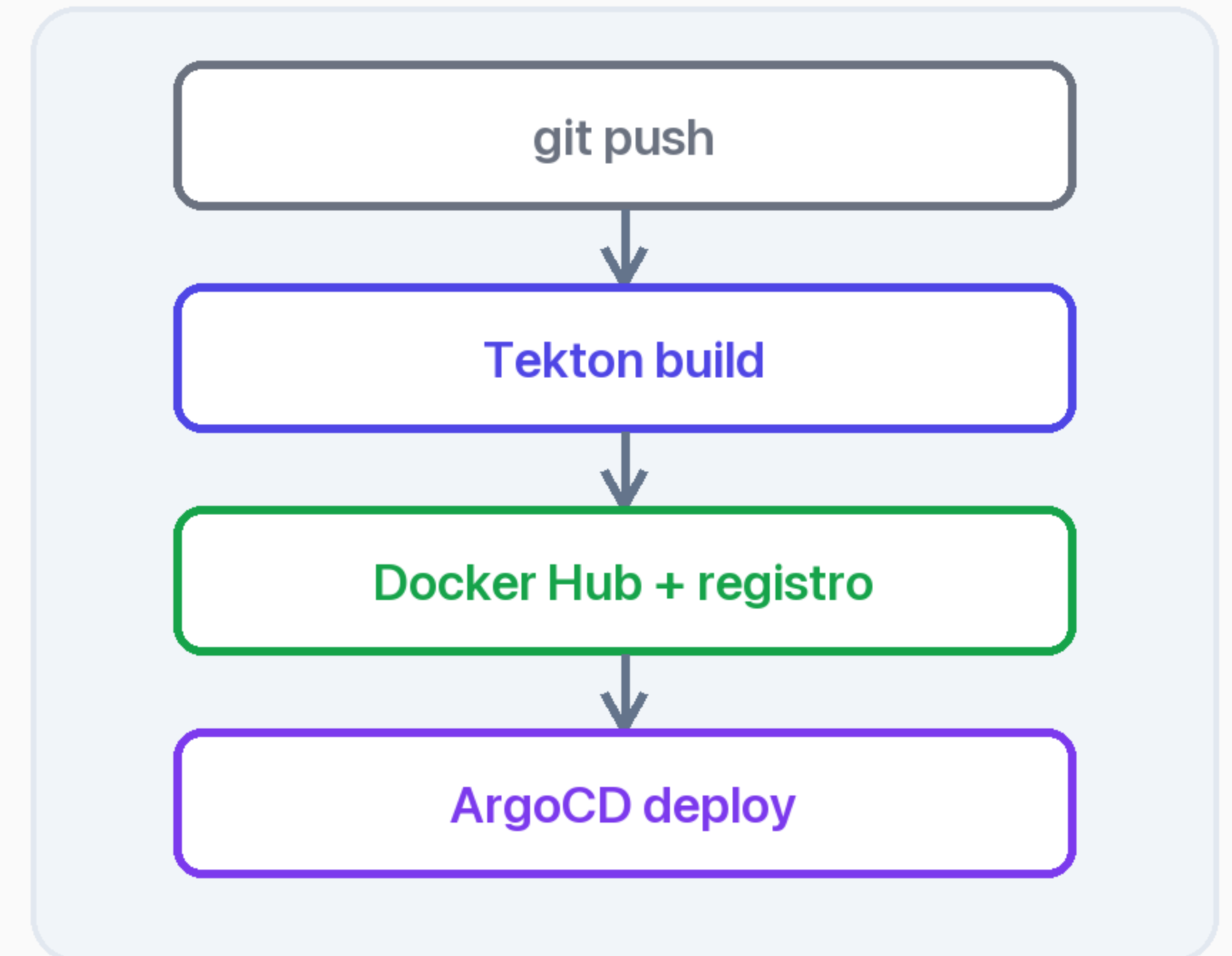
- **Mide:** la longitud de la cola Redis.
- **Sube:** más pods del worker si se acumula.
- **Baja:** a 1 cuando la cola se vacía.
- **Por qué:** picos sin desperdiciar recursos.





# CI/CD — Tekton + ArgoCD

- **Tekton:** con git push, construye la imagen.
- **Doble push:** registro local + Docker Hub.
- **ArgoCD:** despliega solo desde Git (selfHeal).
- **Por qué:** reproducible y auditable.



DESPLIEGUE SIN DOWNTIME

# Canary — Argo Rollouts

10% → 100%

la versión nueva sube por pasos; si el error sube, rollback automático

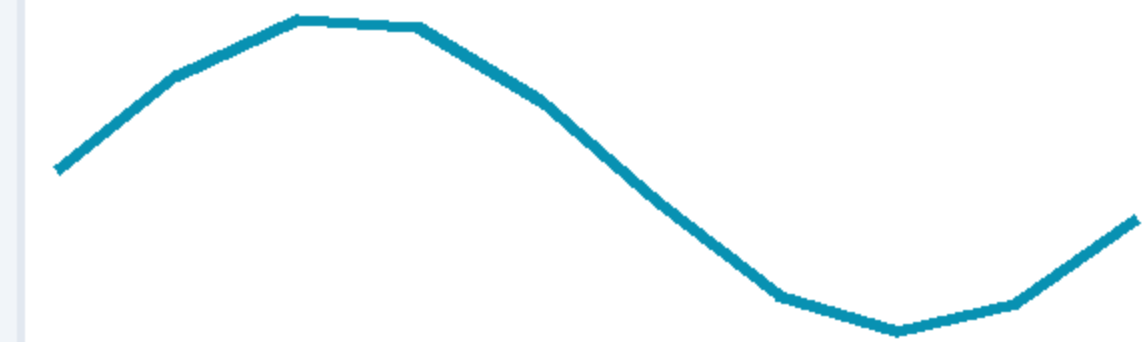


VER QUÉ PASA ADENTRO

# Observabilidad

- **Métricas:** Prometheus + alertas automáticas.
- **Logs:** Loki centraliza todo.
- **Búsqueda:** un pedido por su pedidold.
- **Por qué:** sin medir, no operas.

latencia · cola · error-rate



Prometheus · Loki · Grafana

# Seguridad y gobernanza

- **Ingress + TLS:** HTTPS con cert-manager (CA propia).
- **ResourceQuota:** límites de CPU/memoria por namespace.
- **Secrets:** cifrados con SealedSecrets.
- **Contraseñas:** hash con Argon2.

LO QUE DEMUESTRO EN VIVO

# Resiliencia

3

nodos · sin punto único de falla

0

downtime en el deploy



# Stack en una mirada

## Aplicación

Node.js · Express · EJS · Alpine.js

## Datos

MongoDB 7 (Replica Set) · Redis 7 + BullMQ

## Cluster

kubeadm · Calico (VXLAN) · 3 nodos

## CI/CD

Tekton · ArgoCD · Argo Rollouts

## Escalado

KEDA + HPA

## Observabilidad

Prometheus · Loki · Grafana

## Seguridad

NetworkPolicy · cert-manager · SealedSecrets

# Demo en vivo

---

Pasamos a las 7 demos en el cluster real.

[github.com/SEBASTIANCONTRERAS35/micrositio-pedidos](https://github.com/SEBASTIANCONTRERAS35/micrositio-pedidos)